

EPITA | 2025

iOS & Swift Fundamentals

Antoine Lassoujade, Software Engineer
antoine.lassoujade@epita.fr



ios

iOS - History

iPhone OS **released in 2007**

Running on **arm32 & arm64**

XNU Core for Kernel

System Frameworks executed

Distributed Apps since iPhone OS 2.0

Apps builded with **Xcode**



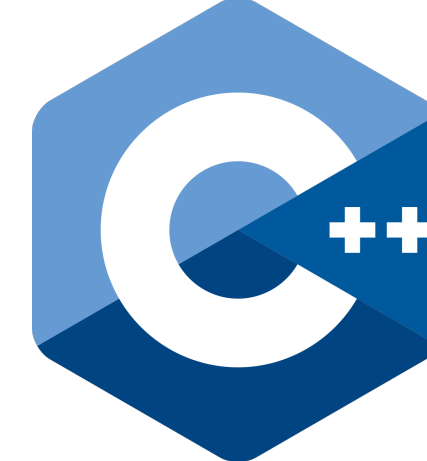
iOS - Languages



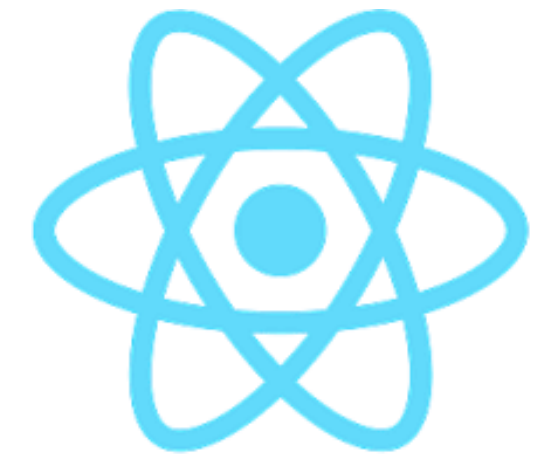
Objective-C



Swift



C/ C++ via Bridging



Cross-Platform

iOS - Languages

Let's compare those two.

iOS - Objective-C



```
#import <Foundation/Foundation.h>

@interface Person : NSObject

@property (nonatomic, strong) NSString *name;

- (void)sayHello;

@end
```


iOS - Objective-C



```
#import "Person.h"
```

```
@implementation Person
```

```
- (void)sayHello {  
    NSLog(@"Hello, my name is %@", self.name);  
}
```

```
@end
```


iOS - Objective-C



```
Person *p = [[Person alloc] init];  
p.name = @"Antoine";  
[p sayHello];
```


Huge, doesn't it ?

iOS - Swift



```
class Person {  
    var name: String  
    init(name: String) {  
        self.name = name  
    }  
    func sayHello() {  
        print("Hello, my name is \(name)")  
    }  
}  
  
let p = Person(name: "Antoine")  
  
p.sayHello()
```


So let's burn Obj-C once for all

Swift

Swift - History

Designed by **Chris Lattner**

Modern & clear syntax

Quick compilation

Interoperability with Obj-C

Object-Oriented Programming

Functional/ Protocols paradigm

Safety-first: **Types**, Immutable ...

Open-Source

Compiled with **LLVM**





Swift - Constants & Variables

let

Immutable

```
let name: String
```

var

Mutable

```
var age: Int
```

Use let **whenever it's possible**

Swift - Computed variables

Do not store a value in memory: **computed when accessed**

Useful to derive values from **stored properties**

Automatically re-evaluated **each time they are read**

Read-only variables

Swift - Computed variables

```
let firstName: String
let lastName: String

var fullName: String {
    "\(firstName) \(lastName)"
}
```

Swift - Types

Swift is **Type-safe: everything is a type**

No primitive types in Swift (char, short, long)

Fundamentals Types, coming from **Standard Library**:

String

Int

Double

Float

Bool

They can be declared with « : » after property's name, or **type inferred**

alt + click to check the type

Swift - Functions

Defined by **a unique name** in its context

Can contain **one or more** parameters

Can **return a Type** or not (aka **Void** Type)

Void is the **implicit return type by default**

```
func myFuncName(myParam: ParamType) -> ReturnType
```

```
 typealias Void = ()
```

Swift - Functions Params

Standard Params

```
func myFunc(p: ParamType)
```

External and Internal Params

```
func myFunc(extP intP: ParamType)
```

Anonymous Params

```
func myFunc(_ intP: ParamType)
```


Swift - Functions Params

Default Params

```
func myFunc(p: ParamType = .init())
```

Variadic Params (transformed in **Array**)

```
func myFunc(params: ParamType...)
```

Inout Params (called with **&**)

```
func myFunc(_ p: inout ParamType)
```

Swift - Classes

Blueprint used to create objects

Regroups **properties (class variables)**, **methods (class functions)**

Reference type

Support **inheritance**, **init** and **deinit**

final to block inheritance (for API design, security and compilation optimization)

static for variables or methods which are not linked with an instance

init have to initialize all properties

deinit can called

Swift - Classes

```
class ClassName {}
```

```
class SubclassName: ClassName {}
```


Swift - Override & Polymorphism

`override`

replace parent's class behavior

`super`

call parent's class behavior

`final`

block inheritance

Swift - Override & Polymorphism

```
class Car {  
    var brand: String  
    var maxSpeed: Int { 200 } // computed  
  
    init(brand: String) {  
        self.brand = brand  
    }  
  
    func description() -> String {  
        return "\(brand) can go up to \(maxSpeed) km/h"  
    }  
}
```

Swift - Override & Polymorphism

```
class ElectricCar: Car {  
    var batteryCapacity: Int  
  
    init(brand: String, maxSpeed: Int, batteryCapacity: Int) {  
        self.batteryCapacity = batteryCapacity  
        super.init(brand: brand, maxSpeed: maxSpeed)  
    }  
  
    override func description() -> String {  
        let base = super.description()  
        return base + " and has a battery of \(batteryCapacity) kWh"  
    }  
}
```


Your Turn !

Create a new **Playground**

Create a `Person` class with the following characteristics: a `firstName`, a `lastName`, a `fullName`, an `id`, an `age`, a `shortDescription()` that returns a short description of the person (for example: `fullName` and `Age`)

Create an `EPITASStudent` class with the following characteristics: It cannot be inherited, It has a property in addition of all the others: `averageRating`, Its `id` reuses the person's `id`, but prefixed with "EPITA-" , It overrides `shortDescription()` to add extra information about the student (for example: the average rating), in addition of the person's `shortDescription()`

Implement a function `celebratesBirthday()` that increases the age of a given `Person`.

Your code should demonstrate polymorphism: Create an array of `Person`, Put both `Person` and `EPITASStudent` instances inside it, Iterate over the array and call `shortDescription()` on each element and observe the different behaviors !

Good luck !

Time's up !

Let's correct

How are you ?

Wrap-up

Swift - Structures

Groups related data and behaviors together (as classes)

Value type: When you assign or pass a struct, you get a copy

Reference type

Doesn't support inheritance

Memberwise Initializer: Swift automatically generates an initializer

Stored directly in memory stack

Swift - Classes vs Structures

`class`

Reference Type

For Complex Datas, Managers etc.

Mutable

Inheritance

`struct`

Value Type

For Simple datas

Immutability by design

Don't support inheritance

Most of Swift Fundamentals Types are **structs**

Swift - Collections: Array

An **ordered** collection of values

Allow **duplicates**

Access using **indexes**

Bunch of incredible features and helping tools !

```
let myArray: Array<Int> = [0, 1, 2]
```


Swift - Collections: Set

An collection of **unique values** without **any order**

No **duplicates**

Fast membership testing ($O(1)$)

Elements must be **Hashable**

```
let fruitBasket: Set<Fruit> = [  
    .apple,  
    .strawberry,  
    .banana  
]
```

Swift - Collections: Dictionary

The Swift equivalent of **HashMap**

Unordered

Unique keys

Complexity: $O(1)$

```
let fruitWithPrice: [String: Int] = [  
    "Apple": 2,  
    "Strawberry": 3,  
    "Banana": 100  
]
```

Swift - Collections Protocols

Arrays, dictionaries and sets all conform to **Collection** and **Sequence**:

forEach

map

filter

reduce

sorted

...

Swift - Generics

To write **flexible** and **reusable code** that works with *any* type, but still preserving strong type safety

Declared with a **placeholder**, **replaced at compile time** with a real type, so **zero runtime cost**

Work with functions, methods, types, protocols

```
struct Array<T>  
func append(newElement: T)  
let alphabet: Array<Character>
```


Your Turn !

1. Create a new **Playground**.

2. Implement a Stack

3. This Stack is able to:

Add an element

Remove the most recently added element

Check whether the **stack is empty**

4. Add a short demonstration showing how your Stack behaves when you add and remove elements

Solution

```
struct Stack {  
    private var items: [Int] = []  
  
    mutating func push(_ item: Int) {  
        items.append(item)  
    }  
  
    mutating func pop() -> Int? {  
        return items.popLast()  
    }  
  
    func peek() -> Int? {  
        return items.last  
    }  
  
    func isEmpty() -> Bool {  
        return items.isEmpty  
    }  
}
```

Swift - Enums

Define a **finite set of related cases, type-safe**

Each case becomes a **distinct value** of the enum type

Cases can store **associated values**

Enums can have **raw values**

Support **methods, computed properties**, and **static** members

Swift - Enums

```
enum Beverage: String {  
    case coffee  
    case tea  
    case juice  
    case custom(name: String)  
    ...  
}
```

Swift - Enums

```
static let defaultDrink: Self = .coffee
```


Swift - Enums

```
var description: String {  
    switch self {  
    case .coffee:  
        return "A hot cup of coffee"  
    case .custom(let name):  
        return "Custom drink: \(name)"  
    }  
}
```

Swift - Operators Cheat Sheet

+ addition

== equal

... closed range

– subtraction

!= not equal

`.. $\infty$` half-open range

* multiplication

< less than

? : ternary

/ division

$\leq$ less or equal

=== same instance

% remainder

> greater than

! == different instance

= simple assignment

>= greater or equal

`+=`, `-=`, `*=`, `/=` compound assignments

! NOT

&& AND

|| OR

Swift - Control Flow - if/else

`if`

Evaluates a **boolean** condition

`else if`

Evaluates a boolean condition **after the previous one**

`else`

Fallback on a given behavior

Swift - Control Flow - if/else

```
let age = 20

if age < 18 {
    print("Minor")
} else if age < 65 {
    print("Adult")
} else {
    print("Senior")
}
```

Swift - Control Flow - if/else

```
if case .coffee = beverage {  
    print("this is coffee")  
} else {  
    print("this is something else")  
}
```


Swift - Control Flow - guard

```
func canEnterInNightClub(age: Int) {  
    guard age >= 18 else {  
        print(« Go back home.»)  
        return  
    }  
    print("Welcome")  
}
```

Swift - Control Flow - switch

More powerful than many languages

Must be **exhaustive**

Supports:

- pattern matching
- ranges
- tuples
- enums
- where

Default for bypassing **unhandled cases**

Swift - Control Flow - switch

```
let beverage = Beverage.coffee

switch beverage {
case .coffee:
    print("this is coffee")
case .tea:
    print("this is tea")

default:
    print("others cases ")
...
}
```


Swift - Control Flow - switch

```
let beverage = Beverage.coffee
```

```
switch beverage {
```

```
case .coffee:
```

```
    print("this is coffee")
```

```
case .tea:
```

```
    print("this is tea")
```

```
case .custom(let custom) where: custom == "beer"
```

```
    print("this is beer !")
```

```
...
```